LAB 9

Code Review Using GitHub Pull Requests for LibaryHub System

Duration	2.5 hours
Objective	Perform structured, professional code reviews using GitHub Pull Requests, applying a review checklist and providing constructive, actionable feedback.
Prerequisites	Completed Labs 1–8; A body of code across LibraryHub to review
Tools / Software	GitHub Pull Requests, A peer or self-review partner

Learning Outcomes

By the end of this lab, students will be able to:

- Apply a structured code-review checklist covering correctness, readability, tests, and design
- Write specific, actionable, and kind review comments
- Use GitHub review features: suggested changes, review states (Approve/Request changes/Comment)
- Understand the reviewer's responsibility as a quality gate

Background and Theory

Code review is one of the highest-leverage quality practices available: studies consistently show it catches defects that automated testing alone misses, especially design and readability issues. An effective review checks (a) correctness — does it do what it claims, including edge cases; (b) tests — are they present, meaningful, and not just asserting the implementation; (c) readability — naming, structure, comments where needed; (d) design — does it fit the existing architecture, is there needless duplication; and (e) risk — security, performance, or backwards-compatibility concerns.

Feedback quality matters as much as coverage. 'This is wrong' is not actionable; 'borrow_book allows a 6th concurrent loan through this branch — should this be > 5 instead of >= 5?' is specific, and lets the author verify the claim themselves. GitHub's suggestion blocks (```suggestion fenced code) let a reviewer propose an exact diff the author can accept with one click.

Pre-Lab Setup

1. Prepare a review checklist in docs/code-review-checklist.md with at least 10 checkpoints across the five categories in the background section.
2. If working with a partner, agree on who authors and who reviews each PR; if working solo, deliberately introduce 3–4 issues into a new branch's code for you to catch in self-review 'as a stranger'.

Worked Example — Reviewing a Deliberately Flawed PR

1. On branch feature/member-loyalty-percentile, implement a borrow_percentile(member, library) function with an intentional off-by-one, a missing docstring, and no test for a library with only one member.
2. Open the PR, then review it against the checklist, leaving inline comments and one suggestion block.

Example review comments

````
# Reviewer's suggestion block on the off-by-one line:
```suggestion
return (rank_index) / (total_members - 1) * 100 if total_members > 1 else 100.0
```
Comment: This currently divides by total_members, which under-counts the top
percentile. Also: please add a docstring and a test for a 1-member library.
````

Practical Exercise Tasks

This lab is designed for 2.5 hours of hands-on work. Complete the following tasks in order; each builds on the previous one and on earlier labs.

Task 1 — Build the Checklist (20 min)

- Finalise docs/code-review-checklist.md with checkpoints for correctness, tests, readability, design, and risk (2+ items each).

Task 2 — Author a Flawed PR (40 min)

- Implement borrow_percentile(member, library) as in the worked example, deliberately including at least 3 issues (a logic bug, a missing edge case, and a style/readability problem), and open the PR.

Task 3 — Full Structured Review (50 min)

- Review the PR against every checklist item, leaving at least 6 inline comments (mix of questions, requests for change, and at least one suggestion block).
- Submit the review as 'Request changes' with a clear top-level summary of the required fixes.

Task 4 — Address the Review (35 min)

- As the author, respond to every comment, push fixes, and accept at least one suggestion block directly through GitHub's UI.
- Re-request review; as the reviewer, verify each fix and submit 'Approve'.

Task 5 — Review Retrospective (25 min)

- In docs/code-review-checklist.md, add a short 'lessons learned' section: which checklist item caught the most valuable issue, and one checklist item you'd add for next time.

Deliverables / Submission Checklist

- docs/code-review-checklist.md with the completed checklist and retrospective
- The merged PR showing the full review conversation (comments, a suggestion block, Request Changes → Approve states)
- Final borrow_percentile() implementation with its bug fixed and a docstring/edge-case test added

LAB 9

Code Review Using GitHub Pull Requests for GradeBook System

Duration	2.5 hours
Objective	Perform structured, professional code reviews using GitHub Pull Requests, applying a review checklist and providing constructive, actionable feedback.
Prerequisites	Completed Labs 1–8; A body of code across GradeBook to review
Tools / Software	GitHub Pull Requests, A peer or self-review partner

Learning Outcomes

By the end of this lab, students will be able to:

- Apply a structured code-review checklist covering correctness, readability, tests, and design
- Write specific, actionable, and kind review comments
- Use GitHub review features: suggested changes, review states (Approve/Request changes/Comment)
- Understand the reviewer's responsibility as a quality gate

Background and Theory

Code review is one of the highest-leverage quality practices available: studies consistently show it catches defects that automated testing alone misses, especially design and readability issues. An effective review checks (a) correctness — does it do what it claims, including edge cases; (b) tests — are they present, meaningful, and not just asserting the implementation; (c) readability — naming, structure, comments where needed; (d) design — does it fit the existing architecture, is there needless duplication; and (e) risk — security, performance, or backwards-compatibility concerns.

Feedback quality matters as much as coverage. 'This is wrong' is not actionable; 'add_score allows score=101 through this branch — should this be > 100 instead of >= 100?' is specific, and lets the author verify the claim themselves. GitHub's suggestion blocks (```suggestion fenced code) let a reviewer propose an exact diff the author can accept with one click.

Pre-Lab Setup

1. Prepare a review checklist in docs/code-review-checklist.md with at least 10 checkpoints across the five categories in the background section.
2. If working with a partner, agree on who authors and who reviews each PR; if working solo, deliberately introduce 3–4 issues into a new branch's code for you to catch in self-review 'as a stranger'.

Worked Example — Reviewing a Deliberately Flawed PR

1. On branch feature/percentile-rank, implement a percentile(student, roster) function with an intentional off-by-one, a missing docstring, and no test for an empty roster.
2. Open the PR, then review it against the checklist, leaving inline comments and one suggestion block.

Example review comments

````
# Reviewer's suggestion block on the off-by-one line:
```suggestion
return (rank_index) / (total_students - 1) * 100 if total_students > 1 else 100.0
```
Comment: This currently divides by total_students, which under-counts the top
percentile. Also: please add a docstring and a test for a 1-student roster.
````

Practical Exercise Tasks

This lab is designed for 2.5 hours of hands-on work. Complete the following tasks in order; each builds on the previous one and on earlier labs.

Task 1 — Build the Checklist (20 min)

- Finalise docs/code-review-checklist.md with checkpoints for correctness, tests, readability, design, and risk (2+ items each).

Task 2 — Author a Flawed PR (40 min)

- Implement percentile(student, roster) as in the worked example, deliberately including at least 3 issues (a logic bug, a missing edge case, and a style/readability problem), and open the PR.

Task 3 — Full Structured Review (50 min)

- Review the PR against every checklist item, leaving at least 6 inline comments (mix of questions, requests for change, and at least one suggestion block).
- Submit the review as 'Request changes' with a clear top-level summary of the required fixes.

Task 4 — Address the Review (35 min)

- As the author, respond to every comment, push fixes, and accept at least one suggestion block directly through GitHub's UI.
- Re-request review; as the reviewer, verify each fix and submit 'Approve'.

Task 5 — Review Retrospective (25 min)

- In docs/code-review-checklist.md, add a short 'lessons learned' section: which checklist item caught the most valuable issue, and one checklist item you'd add for next time.

Deliverables / Submission Checklist

- docs/code-review-checklist.md with the completed checklist and retrospective
- The merged PR showing the full review conversation (comments, a suggestion block, Request Changes → Approve states)

- Final percentile() implementation with its bug fixed and a docstring/edge-case test added